Modelos de difusión para IA generativa

Nota técnica 07

Modelos de difusión condicionales

Semillero GIDSYC | Universidad del Magdalena
Semillero GIDSYC | 2026-I

Un modelo de difusión condicional no cambia el ruido inicial ni el proceso directo. Cambia la función inversa aprendida: la red de denoising recibe una variable semántica que selecciona la componente de la mezcla que debe reconstruir.

1 Motivación

El notebook 06 formuló el dataset multicategoría como una mezcla de distribuciones. Si la variable de clase se denota por y, la marginal de imágenes puede escribirse como

$$p_{\text{data}}(x) = \sum_{k=1}^{K} \pi_k \, p_{\text{data}}(x \mid y = k).$$

Esta identidad separa dos tareas generativas distintas. Un modelo no condicional intenta aproximar la marginal completa $p_{\text{data}}(x)$. Un modelo condicional intenta aproximar una familia de distribuciones $p_{\text{data}}(x \mid y = k)$, una por cada valor de la condición.

La diferencia no es solo interpretativa. En un DDPM no condicional, la red recibe x_t y t. En un DDPM condicional, la red recibe además y. Por tanto, la función aprendida deja de ser

$$\epsilon_\theta(x_t, t)$$

y pasa a ser

$$\epsilon_\theta(x_t, t, y).$$

La etiqueta no actúa como una explicación textual. En este notebook es una variable discreta que se representa mediante un embedding aprendible.

> **Idea clave 1 (Condicionamiento).** Condicionar un DDPM por clase significa entrenar una red de denoising que use una variable adicional y para seleccionar la componente semántica durante el proceso inverso. El proceso directo de ruido permanece sin cambios.

2 Objeto estadístico

Consideremos un dataset etiquetado

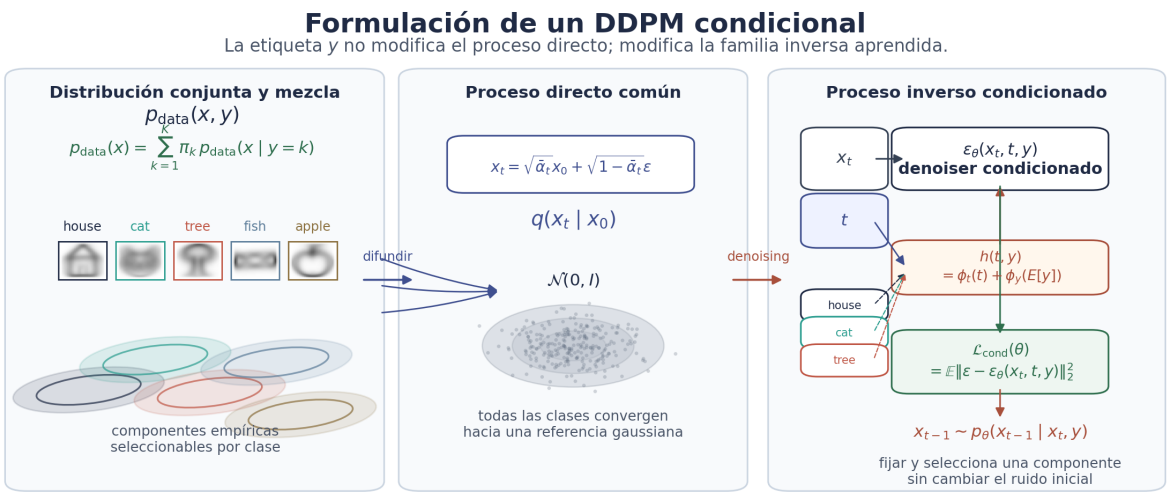
$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}, \qquad x_i \in \mathbb{R}^d, \qquad y_i \in \{1, \dots, K\}.$$

En la implementación del notebook, las clases se almacenan con índices computacionales $0, \dots, K-1$. Matemáticamente usaremos $1, \dots, K$, porque simplifica la notación sin cambiar el objeto representado.

La lectura conjunta del dataset es $p_{\text{data}}(x, y)$. La lectura condicional fija una clase:

$$p_{\text{data}}(x \mid y = k).$$

Si entrenamos un modelo no condicional sobre todas las clases, el modelo observa imágenes de todas las componentes, pero no recibe la información de a cuál pertenecen. Si entrenamos un modelo condicional, cada ejemplo de entrenamiento conserva su etiqueta durante la predicción de ruido.



La figura resume la estructura del notebook. A la izquierda está la mezcla etiquetada. En el centro aparece el proceso directo común, que transforma imágenes de cualquier clase en variables ruidosas. A la derecha aparece el cambio central: la red inversa recibe x_t, t y y, y con esa información define la predicción de ruido y el paso inverso.

3 Proceso directo independiente de la clase

El proceso directo de un DDPM se define por una agenda de varianzas $\{\beta_t\}_{t=1}^{T}$, con

$$\alpha_t = 1 - \beta_t, \qquad \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s.$$

La forma cerrada para simular un tiempo t es

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

En forma distribucional,

$$q(x_t \mid x_0) = \mathcal{N}\left(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t) I\right).$$

La etiqueta no aparece en esta ecuación. Esto es intencional. La difusión directa no necesita saber si la imagen es una casa, un gato o una manzana; solo aplica una perturbación gaussiana controlada en el mismo espacio $\mathbb{R}^{784}$. La variable semántica se usa en la dirección inversa, donde el modelo debe decidir qué estructura reconstruir desde una señal ruidosa.

> **Observación 1 (Independencia condicional).** En este diseño, $q(x_t \mid x_0, y) = q(x_t \mid x_0)$. La clase acompaña al ejemplo durante el entrenamiento, pero no modifica el mecanismo que añade ruido.

4 Objetivo de predicción de ruido

Definamos la ley de muestreo usada durante el entrenamiento como

$$\rho(y, x_0, t, \epsilon) = \widehat{\pi}(y)\, \widehat{p}(x_0 \mid y)\, \mathrm{Unif}\{1, \ldots, T\}(t)\, \mathcal{N}(0, I)(\epsilon).$$

Esta notación resume el procedimiento computacional: elegir una clase de acuerdo con el prior empírico, tomar una imagen de esa clase, muestrear un tiempo discreto y generar ruido gaussiano.

El objetivo no condicional de predicción de ruido puede escribirse como

$$\mathcal{L}_{\mathrm{marg}}(\theta) = \mathbb{E}_{(y,x_0,t,\epsilon)\sim\rho}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|_2^2\right],$$

donde x_t se construye con la forma cerrada del proceso directo. Aunque y aparece en la distribución de donde proviene x_0, la red no lo recibe.

El objetivo condicional introduce la etiqueta como argumento de la red:

$$\mathcal{L}_{\mathrm{cond}}(\theta) = \mathbb{E}_{(y,x_0,t,\epsilon)\sim\rho}\left[\|\epsilon - \epsilon_\theta(x_t, t, y)\|_2^2\right].$$

La modificación parece pequeña, pero cambia la familia de funciones que el modelo puede representar. Ahora la red puede asignar respuestas diferentes a una misma imagen ruidosa y un mismo tiempo si la etiqueta cambia.

En términos de implementación, cada mini-lote contiene pares (x_0, y). Para cada par se muestrea un tiempo t, se genera x_t, y se optimiza la pérdida cuadrática entre el ruido real ϵ y la predicción $\epsilon_\theta(x_t, t, y)$.

5 Representación de la condición

Una etiqueta discreta no puede entrar directamente a una red convolucional como si fuera una imagen. Se transforma primero en un vector aprendible. Si hay K clases y usamos dimensión m, la tabla de embeddings es

$$E \in \mathbb{R}^{K \times m}.$$

La clase y selecciona una fila:

$$e_y = E[y] \in \mathbb{R}^m.$$

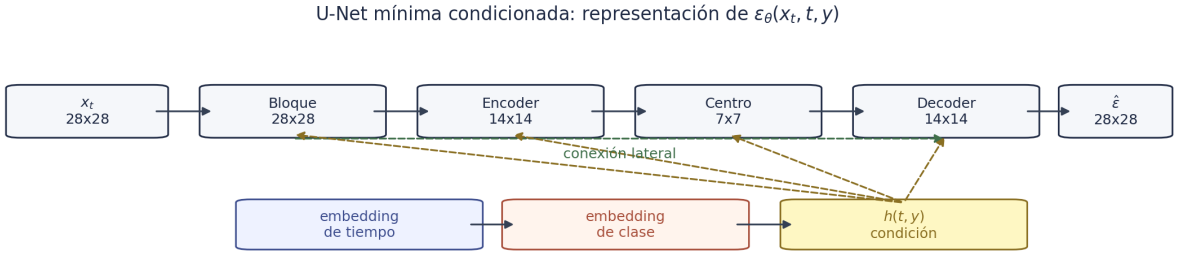
El tiempo también se representa mediante una función de embedding:

$$\phi_t(t) \in \mathbb{R}^m.$$

En el notebook, la representación condicional interna se construye como

$$h(t, y) = \phi_t(t) + \phi_y(e_y).$$

Aquí ϕ_y es una pequeña red feedforward aplicada al embedding de clase. El vector $h(t, y)$ se proyecta dentro de cada bloque convolucional y actúa como un sesgo dependiente de tiempo y clase.

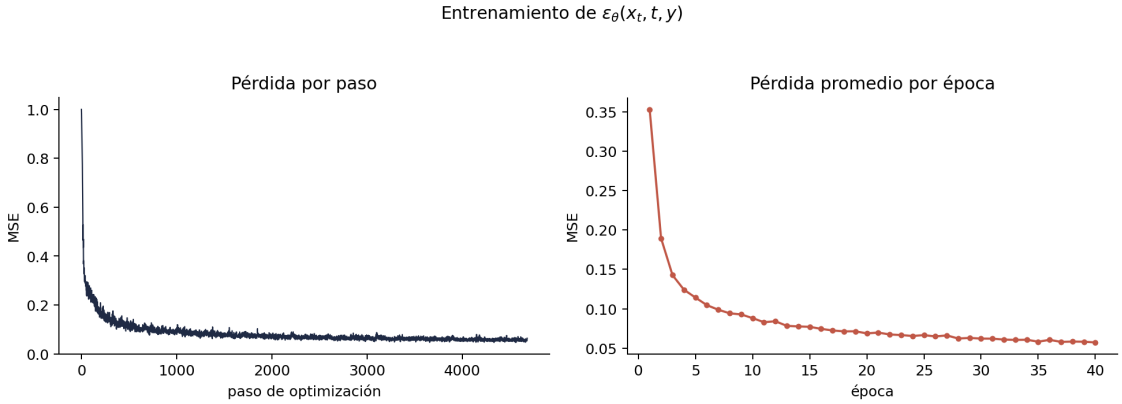


El diagrama muestra la decisión arquitectónica principal. La ruta superior procesa la imagen ruidosa con una U-Net mínima. La ruta inferior construye la condición. La salida de la red tiene la misma forma que la entrada porque el objetivo no es producir directamente una imagen limpia, sino predecir un tensor de ruido $\widehat{\epsilon} \in \mathbb{R}^{1\times 28\times 28}$.

6 Entrenamiento y checkpoint

El experimento usa cinco clases de QuickDraw: *house*, *cat*, *tree*, *fish* y *apple*. Para mantener controlada la mezcla, el entrenamiento toma 6000 muestras por clase. La agenda de difusión usa $T = 500$ pasos y una varianza lineal desde 10^{-4} hasta $2 \cdot 10^{-2}$.

El número de parámetros entrenables de la red es 1,751,009. Es una arquitectura pequeña comparada con modelos generativos modernos, pero suficiente para evidenciar el mecanismo de condicionamiento en imágenes 28×28.



La pérdida decrece porque la red aprende a aproximar el ruido usado para construir x_t. Esta curva no debe interpretarse como una métrica completa de calidad generativa. Una pérdida baja es necesaria, pero no suficiente: el modelo también debe producir trayectorias inversas visualmente coherentes.

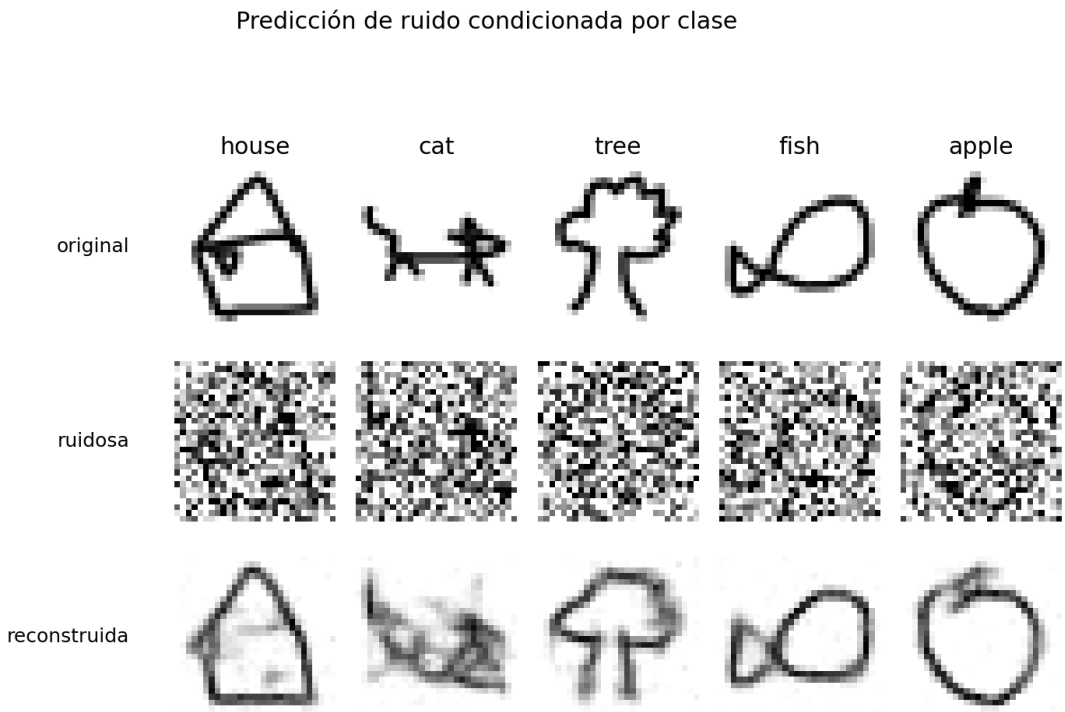
Observación 2 (Reproducibilidad). El notebook guarda un checkpoint con pesos, configuración, nombres de clase e historial de pérdida. Las ejecuciones posteriores cargan ese archivo en lugar de repetir el entrenamiento.

7 Denoising condicionado

Antes de generar desde ruido puro, conviene revisar una tarea más directa. Se toma una imagen real x_0, se perturba hasta un tiempo intermedio t, y se usa la red para predecir el ruido. A partir de esa predicción puede estimarse

$$\widehat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon_\theta(x_t, t, y)}{\sqrt{\bar{\alpha}_t}}.$$

Esta expresión no define todavía una cadena generativa completa. Es una reconstrucción puntual que permite verificar si la red aprendió una operación de denoising compatible con la etiqueta.



La fila superior muestra imágenes reales, la fila intermedia muestra versiones ruidosas y la fila inferior muestra reconstrucciones aproximadas. La recuperación no es perfecta, pero conserva rasgos de la clase. Esta prueba es importante porque examina la función aprendida antes de someterla a una trayectoria inversa completa, donde los errores pueden acumularse.

8 Muestreo inverso condicionado

Para generar, se inicia desde

$$x_T \sim \mathcal{N}(0, I)$$

y se recorre la cadena inversa. En cada paso se fija la misma etiqueta y y se evalúa la red $\epsilon_\theta(x_t, t, y)$. La media DDPM usada en el paso inverso es

$$\mu_\theta(x_t, t, y) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(x_t, t, y) \right).$$

Para $t > 1$, el paso completo incorpora una perturbación gaussiana:

$$x_{t-1} = \mu_\theta(x_t, t, y) + \sqrt{\tilde{\beta}_t}\, z, \qquad z \sim \mathcal{N}(0, I).$$

La varianza posterior usada en esta perturbación es

$$\tilde{\beta}_t = \beta_t \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}.$$

La etiqueta condiciona la media de transición mediante la predicción de ruido. No cambia la distribución inicial x_T.

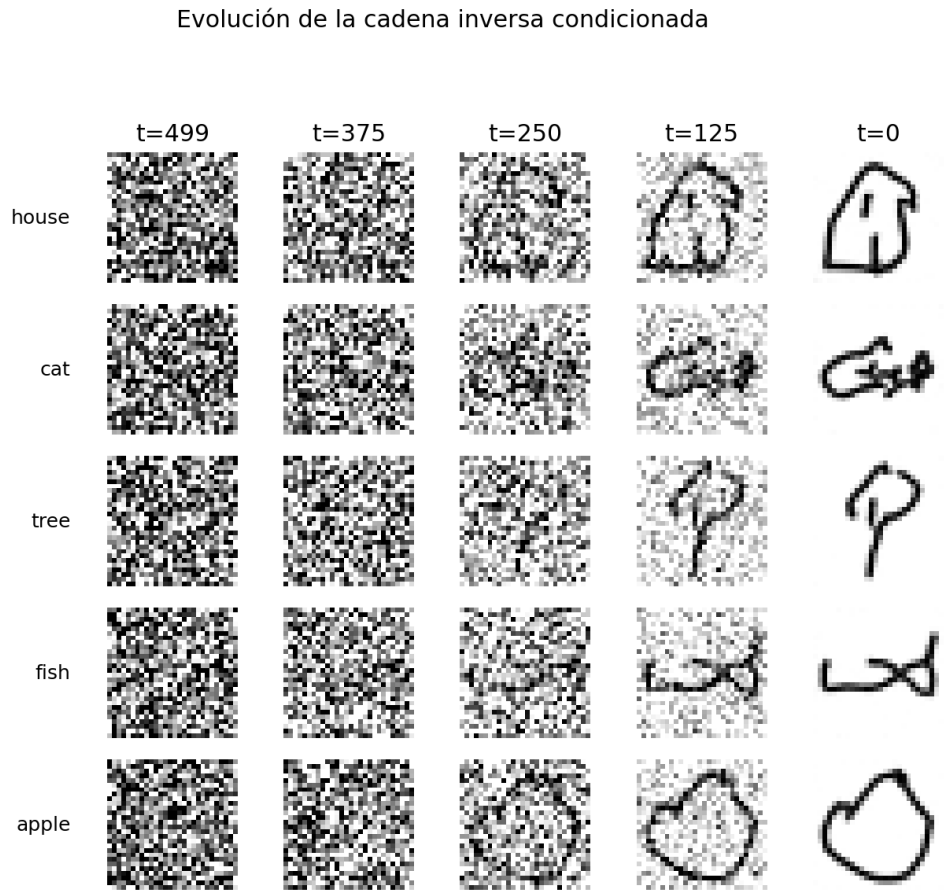
Las muestras generadas muestran que el modelo aprendió una separación semántica básica. Las filas no tienen la misma estructura visual: las casas tienden a formar contornos con techo, los peces cuerpos alargados, las manzanas contornos cerrados con tallo, y los árboles estructuras con tronco o copa. La calidad no es uniforme, pero el objetivo de este notebook no es producir el mejor generador posible; es verificar que el condicionamiento discreto funciona.

9 Trayectoria inversa condicionada

Además de observar muestras finales, es útil examinar estados intermedios de la cadena inversa. Para una etiqueta fija $y = k$, la trayectoria

$$x_T,\ x_{T-1},\ \ldots,\ x_0$$

no debe interpretarse como una interpolación lineal entre ruido e imagen. Cada estado depende de la predicción de ruido de la red, de la agenda de varianza y de la perturbación gaussiana introducida en los pasos intermedios.



La figura muestra que la estructura visual no aparece de una sola vez. En tiempos altos domina el ruido; en tiempos intermedios surgen trazos gruesos y, hacia el final, la muestra adopta una forma compatible con la etiqueta. Esta visualización conecta la formulación iterativa del muestreo con el resultado final observado.

10 Misma semilla, distinta condición

Una prueba más estricta consiste en usar el mismo ruido inicial para varias etiquetas. Sea x_T fijo. Ejecutamos la cadena inversa varias veces, cambiando solo y:

$$x_0^{(k)} \sim p_\theta(x_0 \mid x_T, y = k).$$

Si el modelo ignorara la etiqueta, las salidas tenderían a ser muy similares. Si la usa, la misma semilla inicial puede dar lugar a estructuras semánticas diferentes.

Esta figura es una de las evidencias más claras del notebook. La fuente de ruido es común, pero la condición desplaza la trayectoria hacia clases distintas. Esto muestra que el control generativo no proviene de escoger un ruido inicial especial, sino de modificar la función inversa aplicada en cada paso.

11 Limitaciones

El condicionamiento por etiqueta no resuelve todos los problemas de generación. Primero, la etiqueta es discreta y solo permite seleccionar entre clases conocidas. Segundo, la arquitectura usada es pequeña y no incorpora mecanismos más expresivos como atención. Tercero, el muestreo sigue siendo iterativo y costoso frente a otros procedimientos aproximados. Cuarto, la evaluación sigue siendo cualitativa.

Estas limitaciones no debilitan el resultado conceptual. Al contrario, delimitan el alcance del notebook: mostrar el paso desde una marginal no condicional hacia una familia generativa condicionada por clase.

12 Resumen

Esta nota formalizó el primer DDPM condicional del curso. El proceso directo conserva la misma forma gaussiana:

$$q(x_t \mid x_0, y) = q(x_t \mid x_0).$$

La diferencia aparece en la red inversa:

$$\epsilon_\theta(x_t, t) \quad \longrightarrow \quad \epsilon_\theta(x_t, t, y).$$

Para implementar esa dependencia se usa una tabla de embeddings de clase, combinada con una representación temporal. La condición resultante modula los bloques convolucionales de una U-Net pequeña.

El experimento mostró entrenamiento, reconstrucción puntual y muestreo inverso. Las muestras condicionadas y la prueba con la misma semilla inicial evidencian que el modelo usa y para seleccionar componentes semánticas. Con esto se completa la transición desde generación no condicional hacia generación controlada por etiqueta.

13 Preguntas de estudio

1. ¿Por qué $q(x_t \mid x_0, y) = q(x_t \mid x_0)$ en este notebook?
2. ¿Qué cambia matemáticamente entre $\epsilon_\theta(x_t, t)$ y $\epsilon_\theta(x_t, t, y)$?
3. ¿Qué representa la matriz de embeddings $E \in \mathbb{R}^{K \times m}$?
4. ¿Por qué la pérdida de entrenamiento sigue siendo una MSE de predicción de ruido?
5. ¿Qué información aporta la reconstrucción puntual $\widehat{x}_0$?
6. ¿Por qué la prueba de misma semilla y etiquetas distintas evalúa el uso de la condición?
7. ¿Qué limitaciones tiene un modelo condicionado solo por etiquetas discretas?

14 Continuidad

El siguiente paso del curso es classifier-free guidance. En ese enfoque, el modelo aprende a operar tanto con condición como sin condición. Esa doble capacidad permite combinar predicciones durante el muestreo y controlar la fuerza con la que la etiqueta influye en la generación. La pregunta deja de ser únicamente si el modelo puede obedecer una condición, y pasa a ser qué tan fuerte debe ser esa condición para equilibrar fidelidad, diversidad y control.